SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1005	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	Kanish Kumar V	**Reg. No.:**	192524361
Date:	27.10.26	**Duration:**	60 minutes

Code of Conduct

I Kanish Kumar V (192524361) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Kanish Kumar V

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:

Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

1. **Study and Analyze the Scenario**
   - Carefully understand the given problem statement.
   - Identify the business domain, stakeholders, existing challenges, and system objectives.
2. **Identify Functional and Non-Functional Requirements**
   - List all functional requirements required by the proposed system.
   - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.
3. **Identify Actors and Use Cases**
   - Identify all primary and secondary actors interacting with the system.
   - List the major use cases associated with each actor.
   - Draw a Use Case Diagram representing the interactions between actors and the system.
4. **Prepare the Software Requirement Specification (SRS)**
   Develop an SRS document containing:
   - Introduction
   - Problem Description

- Scope of the System
- Functional Requirements
- Non-Functional Requirements
- Use Case Descriptions
- Data Requirements
- External Interface Requirements
- System Features

5. **Identify Assumptions and Constraints**
   - List the assumptions considered while developing the system.
   - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
6. **Validate Requirement Completeness and Consistency**
   - Verify whether all stakeholder requirements have been addressed.
   - Check for ambiguity, redundancy, inconsistency, and missing requirements.
   - Suggest improvements to enhance the quality and completeness of the requirement specification.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification (25%)	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modelling (20%)	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification (SRS)	SRS is well-structured, complete, professionally organized, and	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or

Documentation (15%)	includes all required sections.			lacks essential components.
Assumptions, Constraints, and Requirement Validation (10%)	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	**Mark**
Scenario Understanding and Problem Analysis (30%)	/5
Functional and Non-Functional Requirement Identification (25%)	/5
Actor Identification and Use Case Modelling (20%)	/5
Software Requirement Specification (SRS) Documentation (15%)	/5
Assumptions, Constraints, and Requirement Validation (10%)	/5
TOTAL	/25

Smart Library Seat Reservation and Study Space Management System

SOFTWARE REQUIREMENT SPECIFICATION (SRS)

1. Introduction

1.1 Purpose

The purpose of this SRS document is to describe the requirements of the Smart Library Seat Reservation and Study Space Management System. It defines the system's features, users, interfaces, data requirements, and operational constraints.

1.2 Scope

The system allows students to view available library seats, reserve seats, check in, check out, and cancel bookings. It helps administrators manage seating facilities, monitor occupancy, detect unused reservations, send notifications, and generate seat utilization reports.

The main goal is to improve study-space utilization, reduce the time spent searching for seats, and provide convenient reservation facilities.

2. Problem Description

Libraries often have limited study spaces, especially during examination periods. Students may waste time searching for vacant seats, while manual management makes it difficult to track reservations and occupancy.

Reserved seats may remain unused when students fail to arrive. Administrators may also find it difficult to enforce time limits and understand seating patterns.

The proposed system provides a centralized platform for managing seat reservations and occupancy. It updates seat status based on booking, check-in, check-out, cancellation, and expiration events.

3. System Objectives

The main objectives are:

- Display available library seats and study spaces.
- Allow students to reserve and cancel seats.
- Manage check-in and check-out.
- Monitor seat occupancy.
- Identify unused reservations and release seats according to library rules.
- Enforce reservation time limits.
- Send booking confirmations and reminders.
- Generate seat utilization reports for better capacity planning.

4. Stakeholders

Stakeholder	Responsibility
Students	View seats, reserve seats, check in, check out, and manage bookings.
Library Administrator	Manage seats, configure reservation rules, and monitor occupancy.
Library Management	Review reports and make decisions about seating capacity.
System Administrator	Maintain system configuration, access, and backups.
Notification Service	Deliver booking confirmations and reminders.

5. Functional Requirements

Functional requirements describe what the system must do.

- **FR1 – User Registration and Login:** The system shall allow students to register and log in securely.
- **FR2 – Seat Availability:** The system shall display available, reserved, occupied, and unavailable seats.
- **FR3 – Seat Search:** Students shall be able to search for seats by study area and time period.
- **FR4 – Seat Reservation:** Students shall be able to reserve available seats for a permitted period.
- **FR5 – Booking Validation:** The system shall prevent conflicting reservations for the same seat and time.
- **FR6 – Booking Management:** Students shall be able to view their current and previous bookings and cancel eligible reservations.
- **FR7 – Check-in and Check-out:** The system shall record arrival and departure times and update seat status.
- **FR8 – Unused Reservation Detection:** The system shall identify reservations without check-in after the configured grace period.
- **FR9 – Automatic Seat Release:** The system shall expire unused reservations and release seats according to the reservation policy.
- **FR10 – Time-Limit Management:** The system shall enforce maximum booking durations and check-in deadlines.
- **FR11 – Notifications:** The system shall send booking confirmations, reminders, and relevant cancellation or expiration alerts.
- **FR12 – Seat Management:** Administrators shall be able to add, update, disable, or remove seat records when permitted.
- **FR13 – Occupancy Monitoring:** Administrators shall be able to view current recorded seat occupancy.
- **FR14 – Reservation Rules:** Administrators shall be able to configure reservation durations, grace periods, and cancellation rules.
- **FR15 – Utilization Reports:** The system shall generate reports showing reservation counts, occupancy patterns, and unused reservations.
- **FR16 – Access Control:** The system shall restrict administrative functions to authorized users.

6. Non-Functional Requirements

Non-functional requirements describe how well the system should operate.

- **Performance:** Seat availability and booking confirmation should normally be displayed within three seconds under standard operating conditions.
- **Security:** Passwords must be stored securely, and unauthorized access to personal information must be prevented.
- **Reliability:** The system must maintain consistent reservation and seat status information.
- **Usability:** The interface should be simple, clear, and easy to use on computers and mobile devices.
- **Scalability:** The system should support an increasing number of users, seats, and reservations.
- **Maintainability:** The application should use modular code so that features can be updated easily.
- **Availability:** The system should be accessible during library operating hours, with appropriate backup and recovery procedures.
- **Privacy:** Personal information should be collected only when necessary and accessed only by authorized users.
- **Compatibility:** The application should work on commonly used modern web browsers.

7. Actors and Use Cases

7.1 Primary Actors

1. **Student:** Registers, logs in, views seats, reserves or cancels bookings, checks in, and checks out.
2. **Library Administrator:** Manages seats and rules, monitors occupancy, identifies unused reservations, and generates reports.

7.2 Secondary Actors

1. **System Administrator:** Maintains technical settings, backups, and system recovery.
2. **Notification Service:** Sends reservation confirmations, reminders, and alerts.

7.3 Use Case Diagram

```
flowchart LR
    Student[Student]
    Admin[Library Administrator]
    SysAdmin[System Administrator]
    Notify[Notification Service]

    subgraph System["Smart Library Seat Reservation and Study Space Management
System"]
        UC1([Register and Log In])
        UC2([View Available Seats])
        UC3([Reserve a Seat])
        UC4([View or Cancel Booking])
        UC5([Check In and Check Out])
        UC6([Receive Notifications])
        UC7([Manage Seats and Rules])
        UC8([Monitor Occupancy])
        UC9([Identify Unused Reservations])
```

```
    UC10([Generate Utilization Reports])
    UC11([Manage System and Backups])
end

Student --- UC1
Student --- UC2
Student --- UC3
Student --- UC4
Student --- UC5
Student --- UC6

Admin --- UC1
Admin --- UC7
Admin --- UC8
Admin --- UC9
Admin --- UC10

SysAdmin --- UC11
Notify --- UC6
```

The diagram represents the main interactions between users and the system. Students use reservation features, administrators manage library operations, and the notification service delivers system messages.

8. Use Case Description

Use Case: Reserve a Seat

- **Actor:** Student
- **Precondition:** The student is logged in and a suitable seat is available.
- **Main Flow:**
    1. The student opens the seat availability page.
    2. The student selects a seat and reservation period.
    3. The system checks seat availability and reservation rules.
    4. The system records the booking.
    5. A confirmation is displayed and a notification is requested.
- **Alternative Flow:** If the seat is unavailable or the requested period is invalid, the system displays an error message.
- **Postcondition:** The reservation is saved successfully without conflicting with another booking.

Use Case: Check-in and Release an Unused Reservation

- **Actor:** Student; the system performs unused-reservation checks automatically.
- **Precondition:** A valid reservation exists.
- **Main Flow:**
    1. The student checks in during the permitted period.
    2. The system records the arrival time and updates the seat status.
    3. If no check-in occurs before the grace period expires, the system identifies the reservation as unused.
    4. The system expires the reservation and releases the seat according to library rules.
- **Postcondition:** Occupancy and reservation status are updated appropriately.

9. Data Requirements

The system must store the following data:

Entity	Important Attributes
User	User ID, name, student ID, email, password hash, role
Study Area	Area ID, area name, floor
Seat	Seat ID, area ID, seat number, seat status
Reservation	Reservation ID, user ID, seat ID, start time, end time, status
Check-in/Check-out	Event ID, reservation ID, arrival time, departure time
Notification	Notification ID, user ID, notification type, delivery status
Reservation Rules	Maximum duration, grace period, operating hours, cancellation policy

Each reservation must be linked to a valid user and seat. The system must prevent overlapping bookings and ensure that seat status is updated correctly when a reservation is cancelled, completed, or expired. Passwords must never be stored as plain text.

10. External Interface Requirements

- **User Interface:** Registration, login, seat availability, booking history, check-in/check-out, and administrator pages.
- **Database Interface:** Stores user, seat, reservation, and occupancy information.
- **Communication Interface:** Uses HTTPS for secure communication when deployed.
- **Notification Interface:** Connects to an email or SMS service to deliver booking updates.
- **Hardware Interface:** The initial system requires a computer or mobile device with a browser. Physical occupancy sensors are optional future enhancements.

11. System Features

The main system features are:

1. **Seat Availability Management:** Displays seats and their current recorded status.
2. **Reservation Management:** Supports booking, viewing, and cancellation.
3. **Occupancy Management:** Records check-in and check-out and updates seat status.
4. **Unused Reservation Detection:** Identifies missed check-ins and releases seats according to configured rules.
5. **Notification Management:** Sends confirmations, reminders, and relevant alerts.
6. **Administrative Management:** Allows authorized staff to manage seats and reservation policies.
7. **Reporting:** Generates utilization statistics to support library planning.

12. Assumptions and Constraints

Assumptions

- Students have access to the internet and a supported web browser.
- Every library seat has a unique identifier.
- Administrators maintain accurate seat information.
- Library management defines reservation duration and check-in rules.
- The initial system relies on recorded check-in/check-out actions rather than automatic sensors.

Constraints

- **Technical:** The system requires a database and application server.
- **Operational:** Students must check in and check out correctly for occupancy records to remain accurate.
- **Economic:** Development, hosting, and notification costs must fit the available budget.
- **Legal:** Student information must be protected according to applicable privacy requirements.
- **Time:** Development must be completed within the assigned academic schedule.

13. Requirement Validation

The requirements cover the main needs of students, library administrators, and library management. The functional requirements address seat searching, reservations, check-in/check-out, unused reservations, notifications, and reporting.

The following checks should be performed:

- **Completeness:** Ensure all required system features have corresponding requirements.
- **Consistency:** Prevent conflicting reservations and incorrect seat-status updates.
- **Ambiguity:** Clearly define the check-in grace period, maximum reservation duration, and cancellation policy.
- **Redundancy:** Keep each requirement focused on a distinct system function.
- **Testability:** Verify each requirement using appropriate functional or performance tests.

Before implementation, library management should confirm the reservation rules, notification method, expected number of users, and data-retention policy.

14. Conclusion

The Smart Library Seat Reservation and Study Space Management System provides an organized method for reserving and managing library seats. It reduces the time students spend searching for available spaces and helps prevent reservation misuse.

The SRS defines the functional and non-functional requirements, actors, use cases, data, interfaces, assumptions, and constraints. Following these requirements will help the development team build a secure, reliable, and user-friendly system that improves library space utilization and supports effective capacity planning.